Practice · Bubble sort

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What bubble sort compares

In one pass of bubble sort, which two elements are compared each time?

- The first and the last.
- **Right answer:** Two neighbours: `a[j]` and `a[j + 1]`.
- The smallest and the largest.
- Each element and `a[0]`.

Feedback: Bubble sort walks from the left and compares each element with its right neighbour. If they are in the wrong order, it swaps them. Class 10, slides 48 to 52.

(mc) L1 · Why temp

Why does a swap need a third variable, `temp`?

- Because C has no `=` for array elements.
- **Right answer:** Because the first assignment overwrites one of the two values, so it must be saved first.
- Because `temp` makes the program faster.
- It does not: `a[0] = a[1]; a[1] = a[0];` also swaps.

Feedback: `a[0] = a[1];` puts the second value into `a[0]`, and the first value is gone. `temp` keeps it safe: save, copy, restore. Class 10, slides 44 to 47.

(mc) L1 · After one pass

After **one** pass of bubble sort (from small to large), what is sure?

- The array is sorted.
- The smallest element is at the start.
- **Right answer:** The largest element is at the end.
- Nothing has moved.

Feedback: Each swap carries the largest value one step to the right, so after one pass it is at the end, in its place. The rest may still be out of order. Class 10, slides 51 and 53.

(mc) L2 · A swap without temp

```
int a[2] = {3, 7};
a[0] = a[1];
a[1] = a[0];
printf("%d %d\n", a[0], a[1]);
```

What does it print?

- `7 3`
- `3 7`
- **Right answer:** `7 7`

- `3 3`

Feedback: The first line puts 7 into `a[0]`, and the 3 is lost. The second line copies that 7 back into `a[1]`. Class 10, slides 44 and 45.

(mc) L2 · One pass of 3 1 2

The array is `3 1 2`. What is it after one pass of bubble sort?

- **Right answer:** `1 2 3`
- `1 3 2`
- `3 1 2`
- `2 1 3`

Feedback: 3 and 1: swap, `1 3 2`. 3 and 2: swap, `1 2 3`. Here one pass happens to sort it all; pass 2 changes nothing. Class 10, slides 49 to 52.

(mc) L2 · How many passes

`bubble_sort` runs the outer loop `for (int pass = 1; pass < n; pass++)`. How many passes does it make on an array of 6 numbers?

- 6
- **Right answer:** 5
- 4
- 36

Feedback: `pass` goes 1, 2, 3, 4, 5: that is n - 1 passes. After 5 passes, five numbers are in their places, and the last one is first by itself. For `6 5 4 3 2 1`, the fifth pass is the one that gives `1 2 3 4 5 6`. Class 10, slides 55 and 56.

(fib) L3 · Finish the swap

`x` is 3 and `y` is 7. Complete the swap so that `x` becomes 7 and `y` becomes 3. Type only what goes after `y =`.

```
int temp = x;
x = y;
y = [temp];
```

Feedback: `temp` kept the old `x`, 3, before `x = y;` changed it. Now `y` gets that 3 back from `temp`. Without the third box, the 3 would be lost and both would be 7. Class 10, slides 44 to 47.

(fib) L3 · One pass of 2 5 1 4

Do **one** pass of bubble sort, from small to large, on `2 5 1 4`. Type the four numbers after the pass, with one space between them.

After one pass: [2 1 4 5]

Feedback: Compare each pair from the left. 2 and 5: in order. 5 and 1: swap, `2 1 5 4`. 5 and 4: swap, `2 1 4 5`. The largest number, 5, has reached the end. Class 10, slides 49 to 51.

(fib) L3 · Passes for 8 numbers

`bubble_sort` makes n - 1 passes. How many passes does it make on an array of 8 numbers? Type only the number.

Passes: [7]

Feedback: Each pass puts one more number in its place at the end. After 7 passes, 7 numbers are in place, and the last one, the smallest, is then in place too. So 8 numbers need 8 - 1 = 7 passes. Class 10, slides 53 to 56.